

Case Study

14. Continuous Integration with Simple Code Analysis

- **Concepts Used:** Jenkins, AWS Cloud9, and SonarQube.
 - **Problem Statement:** "Set up a Jenkins pipeline using AWS Cloud9 to perform a simple code analysis on a JavaScript file using SonarQube."
 - **Tasks:**
 - Create a Jenkins job using AWS Cloud9.
 - Configure the job to integrate with SonarQube for basic code analysis.
 - Run the Jenkins job with a JavaScript file and review the analysis report.
-

1. Introduction

Case Study Overview : This case study is about **Continuous integration (CI)** of pipeline using **Jenkins** and **SonarQube** on **AWS Cloud9**. This setup is used to ensure that analysis of code (Here JavaScript) is automated during the development stage.

Here we have used an **EC2** instance because AWS has stopped giving access to new users from **25th July 2024**. Along with this we have used **GitHub** for code storage and version control.

This Key Feature and Application : The main feature of this case study is to automate the build process with Jenkins combined with SonarQube for analysis. When a new commit is initiated on Github it triggers the Build Now process on Jenkins.

2. Step-by-Step Explanation

Allow the following inbound rules on EC2 instance of Jenkins and SonarQube:

- **HTTP (port 80)**: For accessing Jenkins and SonarQube.
- **SSH (port 22)**: For secure shell access and SonarQube.
- **Custom TCP (port 8080)**: For accessing Jenkins.
- **Custom TCP (port 9000)**: For accessing sonarqube.

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group rule ID	Type	Protocol	Port range	Source	Description - optional	
sgr-09800e2a77808c1eb	HTTP	TCP	80	Custom	Q	Delete
sgr-0f32134e6817c3613	Custom TCP	TCP	9000	Custom	Q	Delete
sgr-0e5948858dd3e6b03	SSH	TCP	22	Custom	Q	Delete
sgr-005c2ebfd0449e859	Custom TCP	TCP	8080	Custom	Q	Delete

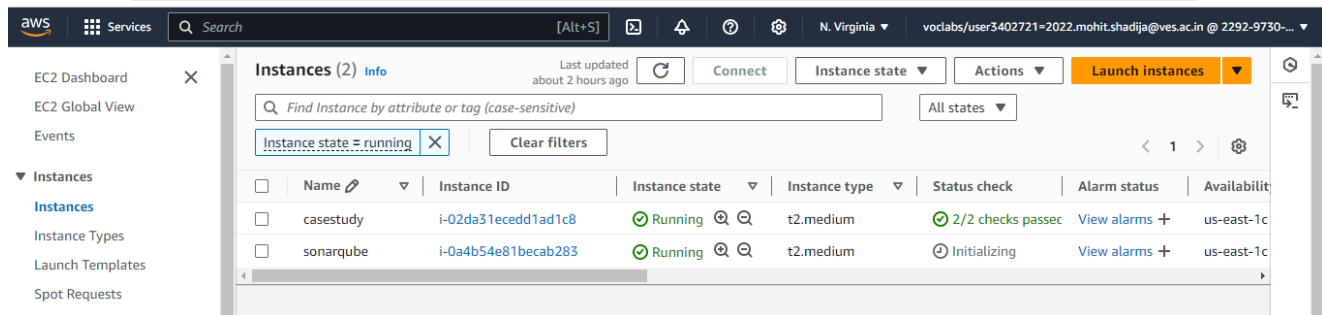
[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

Step 1: Initial Setup and Configuration

1. Launch a **t2.medium** EC2 instance with **Ubuntu**.
2. SSH into the instance using a terminal with the command



Step 2: Install Jenkins on EC2 (Ubuntu)

- `ssh -i path/to/your-key.pem ubuntu@<your-EC2-IP>`
- `sudo apt update`

```
ubuntu@ip-172-31-83-94: ~
ubuntu@ip-172-31-83-94:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [597 kB]
Get:14 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [146 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [114 kB]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [10.2 kB]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [705 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [209 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [305 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [19.8 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [388 kB]
Get:22 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [74.8 kB]
Get:23 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [14.7 kB]
Get:25 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [3820 B]
Get:26 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:27 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [552 B]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [208 B]
```

- `sudo apt install fontconfig openjdk-17-jre`
- `java -version`

```
ubuntu@ip-172-31-83-94:~$ sudo apt install fontconfig openjdk-17-jre
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
fontconfig is already the newest version (2.15.0-1.1ubuntu2).
openjdk-17-jre is already the newest version (17.0.12+7-1ubuntu2~24.04).
0 upgraded, 0 newly installed, 0 to remove and 27 not upgraded.
ubuntu@ip-172-31-83-94:~$
```

```
ubuntu@ip-172-31-83-94:~$ java --version
openjdk 17.0.12 2024-07-16
OpenJDK Runtime Environment (build 17.0.12+7-Ubuntu-1ubuntu224.04)
OpenJDK 64-Bit Server VM (build 17.0.12+7-Ubuntu-1ubuntu224.04, mixed mode, sharing)
```

Add the Jenkins repository

- `sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \`
`https://pkg.jenkins.io/debian/jenkins.io-2023.key`
- `echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]"`
`\ https://pkg.jenkins.io/debian binary/ | sudo tee \`
`/etc/apt/sources.list.d/jenkins.list > /dev/null`

```
ubuntu@ip-172-31-83-94:~$ sudo wget -O /usr/share/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian/jenkins.io-2023.key
--2024-10-23 10:48:10-- https://pkg.jenkins.io/debian/jenkins.io-2023.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.34.133, 2a04:4e42:79::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.34.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3175 (3.1K) [application/pgp-keys]
Saving to: '/usr/share/keyrings/jenkins-keyring.asc'

/usr/share/keyrings/jenkins-keyring.asc 100%[=====>] 3.10K --.-KB/s in 0.001s
2024-10-23 10:48:10 (5.06 MB/s) - '/usr/share/keyrings/jenkins-keyring.asc' saved [3175/3175]
```

- `sudo apt-get update`
- `sudo apt-get install Jenkins`

```
ubuntu@ip-172-31-83-94:~$ sudo apt-get update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Ign:5 https://pkg.jenkins.io/debian-stable binary/ InRelease
Get:6 https://pkg.jenkins.io/debian-stable binary/ Release [2044 B]
Get:7 https://pkg.jenkins.io/debian-stable binary/ Release.gpg [833 B]
Get:8 https://pkg.jenkins.io/debian-stable binary/ Packages [27.9 kB]
Fetched 30.8 kB in 0s (70.4 kB/s)
Reading package lists... Done
```

- `sudo apt-get update`
- `sudo apt-get install Jenkins`

```
ubuntu@ip-172-31-83-94:~$ sudo apt-get install jenkins
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  net-tools
The following NEW packages will be installed:
  jenkins net-tools
0 upgraded, 2 newly installed, 0 to remove and 27 not upgraded.
Need to get 94.4 MB of archives.
After this operation, 96.9 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 net-tools amd64 2.10-0.1ubuntu4 [204
Get:2 https://pkg.jenkins.io/debian binary/ jenkins 2.482 [94.2 MB]
Fetched 94.4 MB in 2s (46.9 MB/s)
Selecting previously unselected package net-tools.
(Reading database ... 82524 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-0.1ubuntu4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4) ...
Selecting previously unselected package jenkins.
Preparing to unpack .../archives/jenkins_2.482_all.deb ...
Unpacking jenkins (2.482) ...
Setting up net-tools (2.10-0.1ubuntu4) ...
Setting up jenkins (2.482) ...
```

- `sudo systemctl start jenkins`
- `sudo systemctl enable jenkins`
- `sudo systemctl status jenkins`

The screenshot shows an AWS CloudShell terminal window with the following content:

```
ubuntu@ip-172-31-83-94:~$ sudo systemctl start jenkins
ubuntu@ip-172-31-83-94:~$ sudo systemctl enable jenkins
Synchronizing state of jenkins.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable jenkins
ubuntu@ip-172-31-83-94:~$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Wed 2024-10-23 10:59:14 UTC; 1min 55s ago
     Main PID: 2280 (java)
       Tasks: 44 (limit: 4676)
      Memory: 742.6M (peak: 744.2M)
         CPU: 16.784s
        CGroup: /system.slice/jenkins.service
               └─2280 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Oct 23 10:59:10 ip-172-31-83-94 jenkins[2280]: 6cf40bcf02ae4c1ca39e94b2bcc089d1
Oct 23 10:59:10 ip-172-31-83-94 jenkins[2280]: This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Oct 23 10:59:10 ip-172-31-83-94 jenkins[2280]: .....
Oct 23 10:59:10 ip-172-31-83-94 jenkins[2280]: .....
Oct 23 10:59:10 ip-172-31-83-94 jenkins[2280]: .....
Oct 23 10:59:14 ip-172-31-83-94 jenkins[2280]: 2024-10-23 10:59:14.133+0000 [id=30] INFO jenkins.InitReactorRunner$1#onAttained: 0
Oct 23 10:59:14 ip-172-31-83-94 jenkins[2280]: 2024-10-23 10:59:14.151+0000 [id=23] INFO hudson.lifecycle.Lifecycle#onReady: Jenk
Oct 23 10:59:14 ip-172-31-83-94 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Oct 23 10:59:14 ip-172-31-83-94 jenkins[2280]: 2024-10-23 10:59:14.170+0000 [id=48] INFO h.m.DownloadService$Downloadable#load: 0>
```

Below the terminal output, the instance ID `i-09d2b962098c77960` (casestudy) is displayed, along with public and private IP addresses.

Open a browser and navigate to *http://<your-EC2-Public-IP>:8080*.

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/lib/jenkins/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

To get Administrator Password

- `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`

Getting Started

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

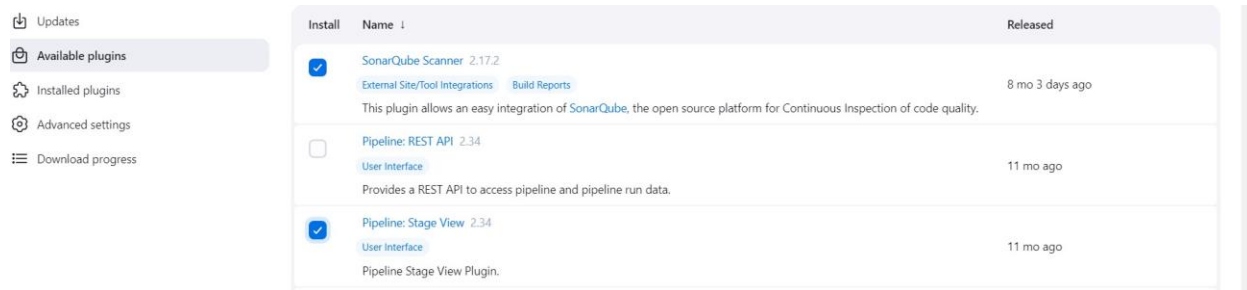
Select plugins to install

Select and install plugins most suitable for your needs.

```
ubuntu@ip-172-31-83-94:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
6cf40bcf02ae4c1ca39e94b2bcc089d1
ubuntu@ip-172-31-83-94:~$
```

1. Install SonarQube Scanner Plugin in Jenkins:

- Go to **Manage Jenkins** → **Manage Plugins**.
- Search for **SonarQube Scanner** and install it.



Step 3: Install Sonarqube in new EC2 (Ubuntu)

1. Prepare your Ubuntu server.

```
sudo apt update

sudo apt upgrade -y
```

2. Install OpenJDK 11 - install java development kit 11 or higher version as now

- `sudo apt install -y openjdk-11-jdk`

3. Install and Configure PostgreSQL

- `sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt/ `lsb_release -cs`-pgdg main" >> /etc/apt/sources.list.d/pgdg.list'`

```
ubuntu@ip-172-31-92-10:~$ sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt/ `lsb_release -cs`-pgdg main" >> /etc/apt/sources.list.d/pgdg.list'
ubuntu@ip-172-31-92-10:~$
```

Add PostgreSQL signing key.

- `wget -q https://www.postgresql.org/media/keys/ACCC4CF8.asc -O - | sudo apt-key add -`

```
ubuntu@ip-172-31-92-10:~$ wget -q https://www.postgresql.org/media/keys/ACCC4CF8.asc -O - | sudo apt-key add -
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
OK
ubuntu@ip-172-31-92-10:~$
```

Install PostgreSQL.

- `sudo apt install -y postgresql postgresql-contrib`

```

ubuntu@ip-172-31-92-10:~$ sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable postgresql
sudo systemctl start postgresql
sudo passwd postgres
su - postgres
createuser sonar
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libcommon-sense-perl libjson-perl libjson-xs-perl libpq5 libtypes-serialiser-perl postgresql-16 postgresql-client-16 postgresql-client-common postgresql
Suggested packages:
  postgresql-doc postgresql-doc-16
The following NEW packages will be installed:
  libcommon-sense-perl libjson-perl libjson-xs-perl libpq5 libtypes-serialiser-perl postgresql postgresql-16 postgresql-client-16 postgresql-client-commo
postgresql-contrib ssl-cert
0 upgraded, 12 newly installed, 0 to remove and 26 not upgraded.
Need to get 17.3 MB of archives.
After this operation, 50.8 MB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libjson-perl all 4.10000-1 [81.9 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-client-common all 257build1.1 [36.4 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 ssl-cert all 1.1.2ubuntu1 [17.8 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-common all 257build1.1 [161 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libcommon-sense-perl amd64 3.75-3build3 [20.4 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libtypes-serialiser-perl all 1.01-1 [11.6 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libjson-xs-perl amd64 4.030-2build3 [83.6 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 libpq5 amd64 16.4-0ubuntu0.24.04.2 [141 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-client-16 amd64 16.4-0ubuntu0.24.04.2 [1271 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-16 amd64 16.4-0ubuntu0.24.04.2 [15.5 MB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql all 16+257build1.1 [11.6 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-contrib all 16+257build1.1 [11.6 kB]
Fetched 17.3 MB in 0s (83.5 MB/s)
Preconfiguring packages ...

```

Enable DB server to start automatically on reboot.

- `sudo systemctl enable postgresql`

Start DB server.

- `sudo systemctl start postgresql`

Change the default PostgreSQL password.

- `sudo passwd postgres`

Switch to the postgres user.

- `su - postgres`

Create a user named sonar.

- `createuser sonar`

Log into PostgreSQL.

- `psql`
- `ALTER USER sonar WITH ENCRYPTED password '<your_password>';`
- `CREATE DATABASE sonarqube OWNER sonar;`
- `GRANT ALL PRIVILEGES ON DATABASE sonarqube to sonar;`

Exit PostgreSQL.

- `\q`

```

No VM guests are running outdated hypervisor (qemu) binaries on this host.
Synchronizing state of postgresql.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable postgresql
New password:
Retype new password:
passwd: password updated successfully
Password:
postgres@ip-172-31-92-10:~$ psql
ALTER USER sonar WITH ENCRYPTED PASSWORD 'my_strong_password';
CREATE DATABASE sonarqube OWNER sonar;
GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;
\q
exit
psql (16.4 (Ubuntu 16.4-0ubuntu0.24.04.2))
Type "help" for help.

postgres=#

```

Return to your non-root sudo user account.

- `exit`

4. Download and Install SonarQube

Install the zip utility, which is needed to unzip the SonarQube files.

- `sudo apt install -y zip`

Locate the latest download URL from SonarQube official download page. At the time of writing this document, the download URL was as follows:

<https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.0.1.46107.zip>

Download the SonarQube distribution files.

- `sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.0.1.46107.zip`

Unzip the downloaded file.

- `sudo unzip sonarqube-9.0.1.46107.zip`

Move the unzipped files to `/opt/sonarqube` directory

- `sudo mv sonarqube-9.0.1.46107 /opt/sonarqube`

```

ubuntu@ip-172-31-92-10:~$ sudo apt install -y zip
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.0.1.46107.zip
sudo unzip sonarqube-9.0.1.46107.zip
sudo mv sonarqube-9.0.1.46107 /opt/sonarqube
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  unzip
The following NEW packages will be installed:
  unzip zip
0 upgraded, 2 newly installed, 0 to remove and 26 not upgraded.
Need to get 350 kB of archives.
After this operation, 933 kB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 unzip amd64 6.0-28ubuntu4.1 [174 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 zip amd64 3.0-13build1 [175 kB]
Fetched 350 kB in 0s (12.6 MB/s)
Selecting previously unselected package unzip.
(Reading database ... 85623 files and directories currently installed.)
Preparing to unpack .../unzip_6.0-28ubuntu4.1_amd64.deb ...
Unpacking unzip (6.0-28ubuntu4.1) ...
Selecting previously unselected package zip.
Preparing to unpack .../zip_3.0-13build1_amd64.deb ...
Unpacking zip (3.0-13build1) ...
Setting up unzip (6.0-28ubuntu4.1) ...
Setting up zip (3.0-13build1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

```

5. Add SonarQube Group and User

Create a sonar group.

- `sudo groupadd sonar`
- `sudo useradd -d /opt/sonarqube -g sonar sonar`
- `sudo chown sonar:sonar /opt/sonarqube -R`

```

ubuntu@ip-172-31-92-10:~$ sudo groupadd sonar
ubuntu@ip-172-31-92-10:~$ sudo useradd -d /opt/sonarqube -g sonar sonar
ubuntu@ip-172-31-92-10:~$ sudo chown sonar:sonar /opt/sonarqube -R
ubuntu@ip-172-31-92-10:~$

```

6. Configure SonarQube

Edit the SonarQube configuration file.

- `sudo nano /opt/sonarqube/conf/sonar.properties`

```

GNU nano 7.2 /opt/sonarqube/conf/sonar.properties
# Property values can:
# - be overridden by environment variables. The name of the corresponding environment variable is the
#   upper-cased name of the property where all the dot('.') and dash('-') characters are replaced by
#   underscores('_'). For example, to override 'sonar.web.systemPasscode' use 'SONAR_WEB_SYSTEMPASSCODE'.
# - be encrypted. See https://redirect.sonarsource.com/doc/settings-encryption.html
#-----
# DATABASE
#
# IMPORTANT:
# - The embedded H2 database is used by default. It is recommended for tests but not for
#   production use. Supported databases are Oracle, PostgreSQL and Microsoft SQLServer.
# - Changes to database connection URL (sonar.jdbc.url) can affect SonarSource licensed products.
#
# User credentials.
# Permissions to create tables, indices and triggers must be granted to JDBC user.
# The schema must be created first.
#sonar.jdbc.username=
#sonar.jdbc.password=
#----- Embedded Database (default)
# H2 embedded database server listening port, defaults to 9092
#sonar.embeddedDatabase.port=9092
#----- Oracle 12c/18c/19c
# The Oracle JDBC driver must be copied into the directory extensions/jdbc-driver/oracle/.
# Only the thin client is supported, and we recommend using the latest Oracle JDBC driver. See
# https://jira.sonarsource.com/browse/SONAR-9758 for more details.
# If you need to set the schema, please refer to http://jira.sonarsource.com/browse/SONAR-5606
#sonar.jdbc.url=jdbc:oracle:thin:@localhost:1521/XE
#----- PostgreSQL 9.6 or greater
#-----

```

Step 1: Find the following lines.

```
#sonar.jdbc.username=
```

```
#sonar.jdbc.password=
```

Step 2: Uncomment the lines, and add the database user sonar and password my_strong_password you created in Section 3.

- `sonar.jdbc.username=sonar`
- `sonar.jdbc.password=my_strong_password`

Step 3: Below those two lines, add sonar.jdbc.url.

- `sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube`

Save and exit the file.

Edit the sonar script file.

- `sudo nano /opt/sonarqube/bin/linux-x86-64/sonar.sh`

locate this line. **#RUN_AS_USER=** Uncomment the line and change it to.

- `RUN_AS_USER=sonar`

Save and exit the file.

7. Setup Systemd Service

- `sudo nano /etc/systemd/system/sonar.service`

Step 1: Paste the following lines to the file.

[Unit]

Description=SonarQube service

After=syslog.target network.target

[Service]

Type=forking

ExecStart=/opt/sonarqube/bin/linux-x86-64/sonar.sh start

ExecStop=/opt/sonarqube/bin/linux-x86-64/sonar.sh stop

User=sonar

Group=sonar

Restart=always

LimitNOFILE=65536

LimitNPROC=4096

[Install]

WantedBy=multi-user.target

Save and exit the file.

Start SonarQube

- `sudo systemctl enable sonar`
- `sudo systemctl start sonar`
- `sudo systemctl status sonar`

Open a browser and navigate to *http://<your-new-EC2-Public-IP>:9000*.

The image displays two side-by-side screenshots of the SonarQube web interface. The left screenshot shows the 'Log In to SonarQube' page, which includes a username input field with the text 'ad', a password input field, and 'Log in' and 'Cancel' buttons. The right screenshot shows the 'Create a project' page. It features two required fields: 'Project display name' and 'Project key', both containing the text 'Sonarqube_CS' and marked with a green checkmark. Below these fields, there is explanatory text about character limits and allowed characters. At the bottom of the 'Create a project' page is a 'Set Up' button.

Step 4: Integrate Jenkins with SonarQube

1. **Generate authentication token**: Generate a token in SonarQube by going to **My Account** → **Security** → **Generate Tokens**.

There's a new version of SonarQube available. Upgrade to the latest active version to access new updates and features. [Learn More](#)

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration More Q

A Administrator

Profile **Security** Notifications Projects

Security

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name Type Expires in

✓ New token "token" has been created. Make sure you copy it now, you won't be able to see it again!

Name	Type	Project	Last use	Created	Expiration
token	Global		Never	October 21, 2024	November 20, 2024

2. Add Credentials in Jenkins:

- Go to **Manage Jenkins** → **Manage Credentials** → **Add a new credential**.
- Add your SonarQube token as a **Secret Text** credential.

3. Configure SonarQube Server in Jenkins:

- Go to **Manage Jenkins** → **Configure System**.
- Find the **SonarQube servers** section and click **Add SonarQube**.
- Enter:
 - **Name:** SonarQube or <any name>
 - **Server URL:** *http://<your-new-EC2-Public-IP>:9000* .
 - **Server authentication token:** Use generated token.

Dashboard > Manage Jenkins > System >

SonarQube installations

List of SonarQube installations

Name

Server URL

Default is http://localhost:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

Advanced ▾

4. Set sonarqube Scanner installer

Manage Jenkins → **Tools** → **SonarQube Scanner** → **Add Installer**

Add SonarQube Scanner

☰ SonarQube Scanner

Name

Sonarqube

☒ Install automatically ?

☰ Install from Maven Central

Version

SonarQube Scanner 6.2.1.4610

▼

Add Installer ▼

Add SonarQube Scanner

Step 5: Create Pipeline project

Enter an item name

CaseStudy

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



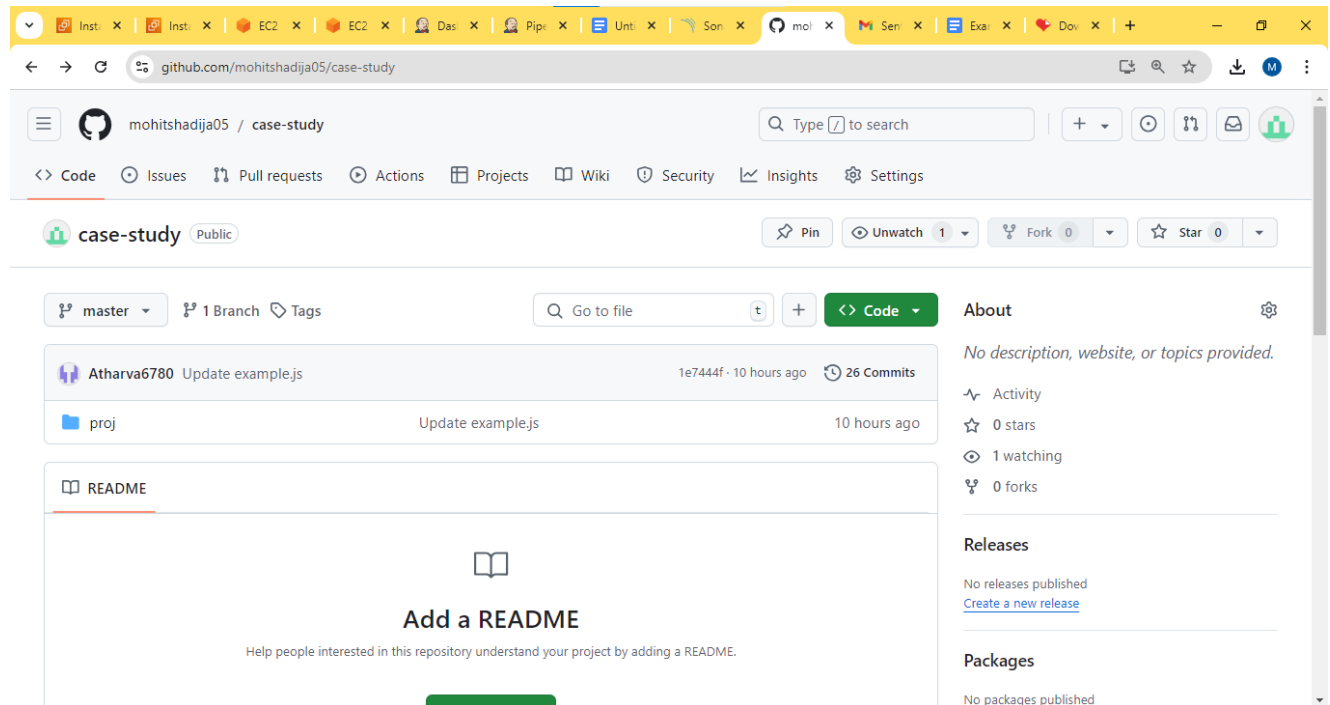
Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different

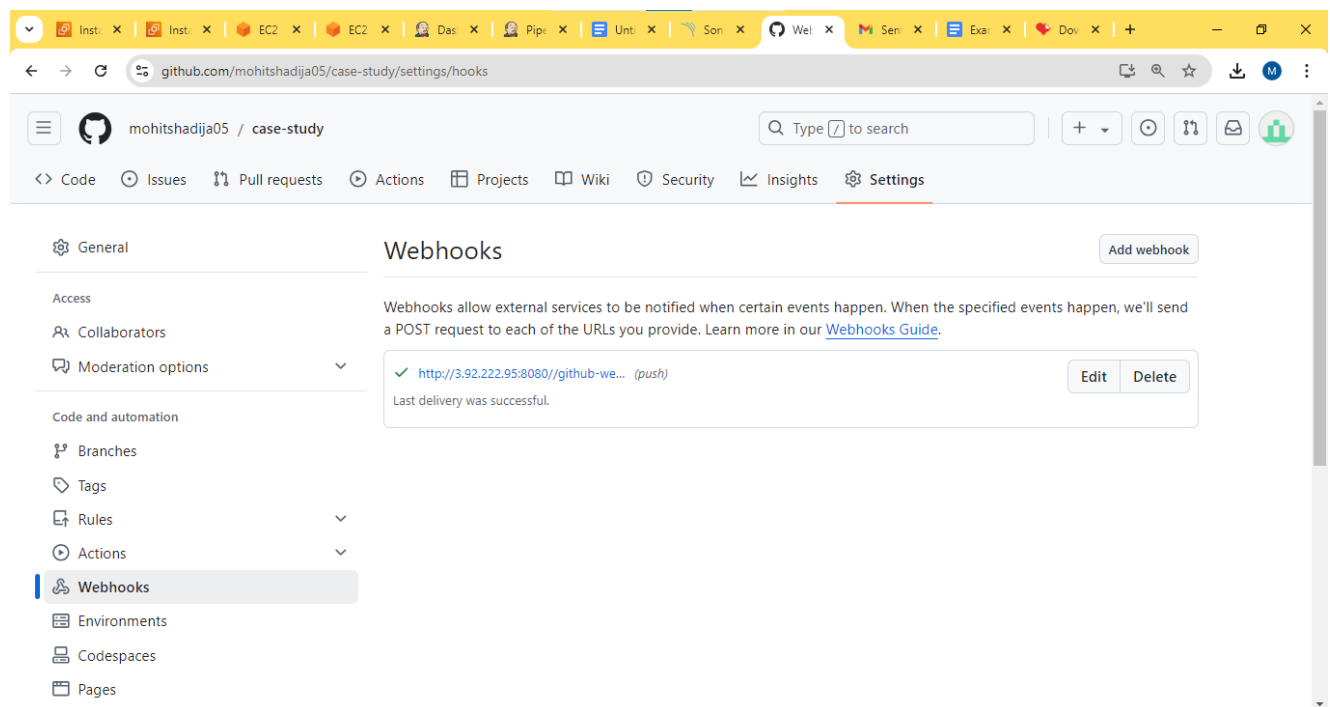
For Continuous Integration:

1) Configure GitHub Webhook:

- Go to your GitHub repository. → Navigate to **Settings > Webhooks** → Click **Add webhook**.



b) Set the Payload URL to: `http://<jenkins url>/github-webhook/`.



c) Choose *application/json* for **Content type**.

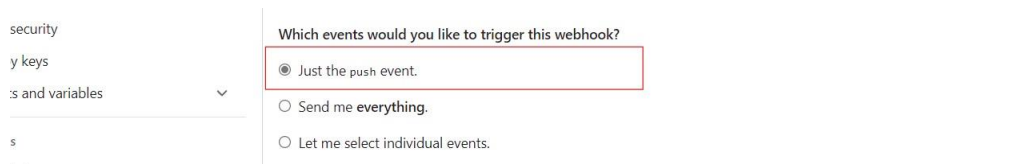


Content type *

application/json

Secret

d) Select **Just the push event**.



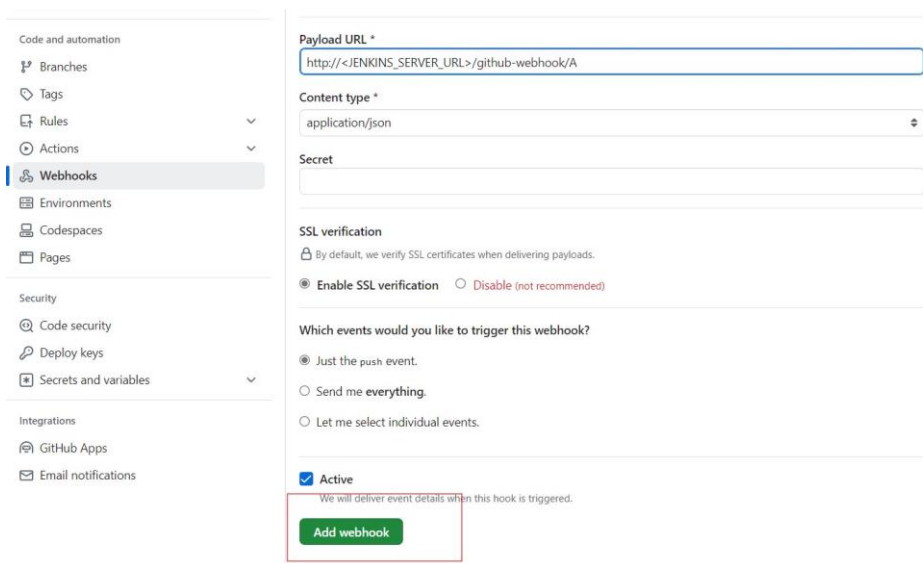
Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me everything.

☐ Let me select individual events.

e) Click **Add webhook**



Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Code security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Payload URL *

http://</ENKINS_SERVER_URL>/github-webhook/A

Content type *

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me everything.

☐ Let me select individual events.

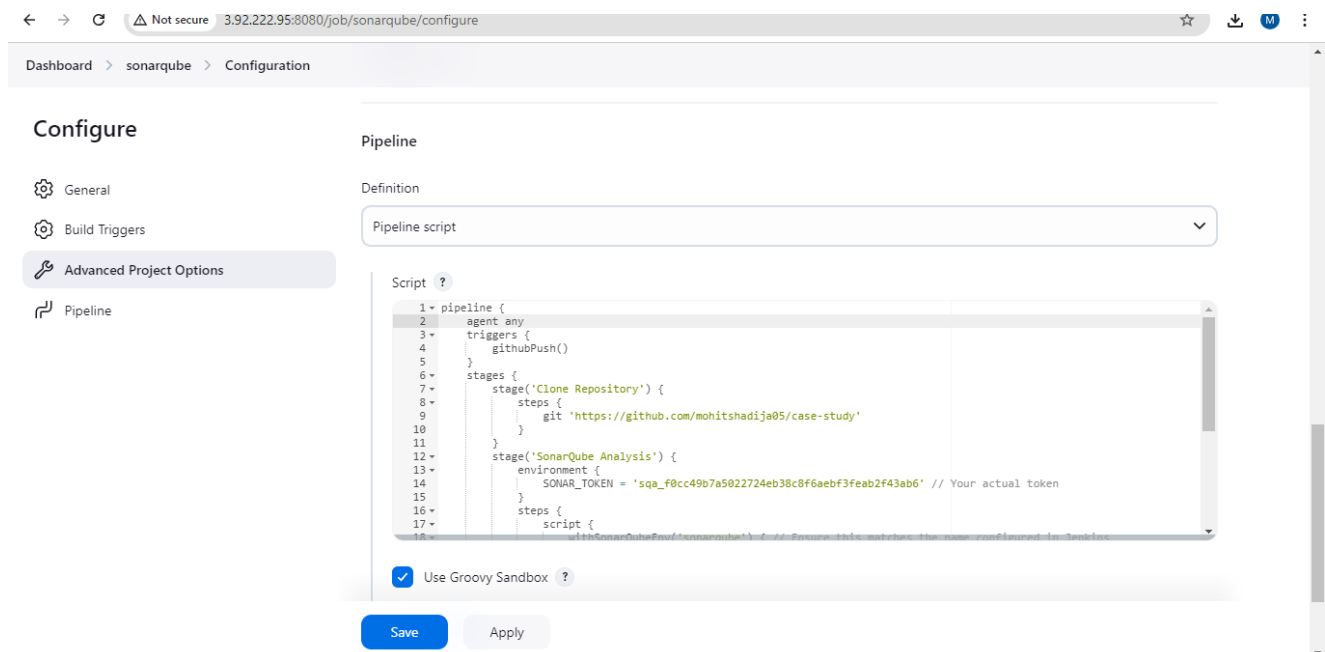
☒ Active

We will deliver event details when this hook is triggered.

Add webhook

Pipeline code:

After adding pipeline : **Save** it and Build project by clicking **Build Now**



Dashboard > sonarqube > Configuration

Configure

General

Build Triggers

Advanced Project Options

Pipeline

Pipeline

Definition

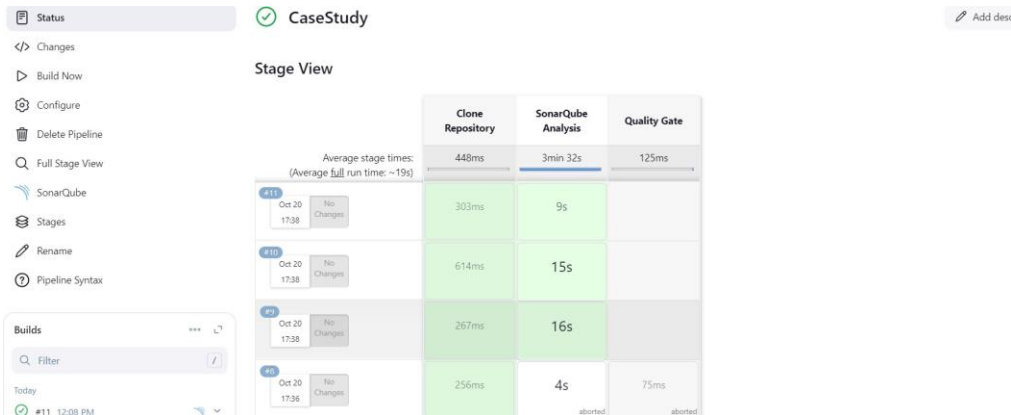
Pipeline script

Script ?

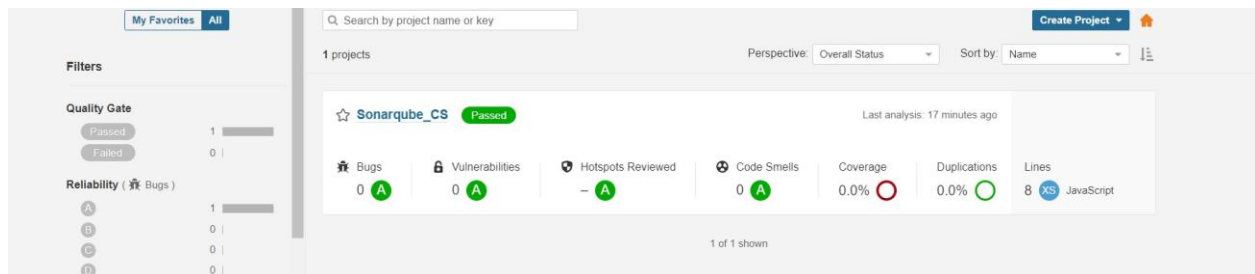
```
1 pipeline {
2   agent any
3   triggers {
4     githubPush()
5   }
6   stages {
7     stage('Clone Repository') {
8       steps {
9         git 'https://github.com/mohitshadija05/case-study'
10      }
11    }
12    stage('SonarQube Analysis') {
13      environment {
14        SONAR_TOKEN = 'sqa_f0cc49b7a5022724eb38c8f6aebf3feab2f43ab6' // Your actual token
15      }
16      steps {
17        script {
18          // SonarQubeEnv (sonarqube) // Ensure this matches the name configured in Jenkins
19        }
20      }
21    }
22  }
23 }
```

☒ Use Groovy Sandbox ?

Save Apply



Sonarqube:



Guidelines:

- Always update your instance (sudo apt update && sudo apt upgrade).
- Use an instance with storage of at least 4GiB RAM and 2 CPU (t2.medium or higher).

3. Key Points:

Jenkins Automation is the process of automatic builds, where Jenkins pulls the code from the GitHub to execute the builds and integrate changes continuously without any interference of humans.

SonarQube Integration, it is a step where SonarQube is integrated on the static level and analyzes the software at the time of building. Bugs, bugs, vulnerabilities, and code smells have been known to improve the quality of the code.

Rather than local deployment we have used an EC2 instance which is reliable, scalable and flexible.

Practice:

- Run through the demo multiple times to ensure everything works smoothly.
- Confirm that Jenkins and SonarQube are running before starting the presentation.
- Check Public IP each time we start presentation because each time new Public IP is allot to instances and according to that configurations are set on Jenkins